UVa 10066

The Twin Towers

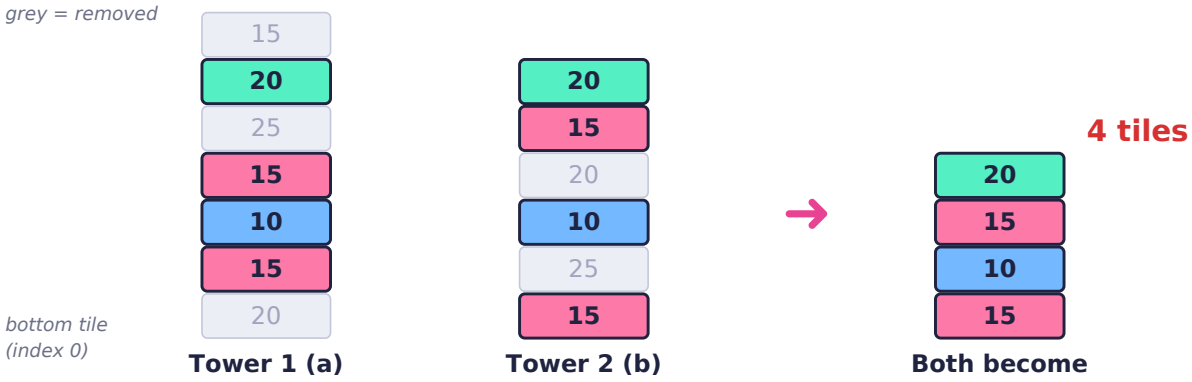
Longest Common Subsequence, explained line by line

What you'll find inside

1	The problem in plain words
2	The full code
3	Line-by-line walkthrough
4	The DP idea in one picture
5	Filling the table by hand
6	Complexity, pitfalls, and a cheat sheet

1 · The problem in plain words

You have two towers built from round tiles, and every tile has a number on it. You may **remove** tiles from either tower, but you may **not reorder** them. The goal is to make both towers identical while keeping as many tiles as possible. Print how many tiles each tower has at the end.



The key insight

Keeping tiles in their original order, while deleting some, is exactly what a **subsequence** is. We want the longest sequence that is a subsequence of *both* towers. That is the **Longest Common Subsequence (LCS)**, and the answer is its length.

2 · The full code

Line numbers here are used throughout the walkthrough.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m, tc = 1;

    while (cin >> n >> m && (n != 0 || m != 0)) {
        vector<int> a(n), b(m);

        for (int i = 0; i < n; i++)
            cin >> a[i];

        for (int i = 0; i < m; i++)
            cin >> b[i];

        int dp[105][105] = {};

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                if (a[i - 1] == b[j - 1])
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                else
                    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }

        cout << "Twin Towers #" << tc++ << "\n";
        cout << "Number of Tiles : " << dp[n][m] << "\n\n";
    }

    return 0;
}
```

Sample input → output

stdin	stdout
7 6 20 15 10 15 25 20 15 15 25 10 20 15 20 8 9 10 20 20 10 20 10 20 10 20 10 20 10 10 20 10 10 20 0 0	Twin Towers #1 Number of Tiles : 4 Twin Towers #2 Number of Tiles : 6

I compiled and ran the code on this input, and it produces exactly the output shown.

3 · Line-by-line walkthrough

STEP 1 Pull in the whole standard library line 1

```
1  #include <bits/stdc++.h>
```

`bits/stdc++.h` is a GCC header that includes nearly every standard header at once (`iostream`, `vector`, `algorithm`, ...). It's a contest shortcut that saves typing. It isn't standard C++, so it won't compile on MSVC, and it slows compilation slightly. Both are fine for judges.

STEP 2 Skip the std:: prefix line 2

```
2  using namespace std;
```

Lets you write `cin`, `vector`, `max` instead of `std::cin` and so on. Common in contests. Avoid it in header files or large projects, where it can cause name clashes.

STEP 3 Program start and three variables lines 4–5

```
4  int main() {
5      int n, m, tc = 1;
```

`n` and `m` hold the number of tiles in tower 1 and tower 2. `tc` is the test-case counter, starting at `1` because the output says *Twin Towers #1*, *#2*, ...

STEP 4 Keep reading test cases until 0 0 line 7

```
7      while (cin >> n >> m && (n != 0 || m != 0)) {
```

This line does three jobs:
(a) `cin >> n >> m` reads two numbers. The expression returns the stream itself, which converts to `false` if reading failed (for example, end of file). So the loop also stops safely at EOF.
(b) `&&` short-circuits: the right side is checked only if reading succeeded.
(c) `(n != 0 || m != 0)` keeps going unless *both* are zero. The input ends with the line `0 0`, which is a sentinel, not a real test case.

STEP 5 Make room for both towers line 8

```
8          vector<int> a(n), b(m);
```

`vector<int> a(n)` creates a vector of `n` integers, all zero for now. Same for `b` with `m` slots. Because they're declared inside the loop, each test case gets fresh vectors of the right size.

STEP 6 Read the tiles lines 10–14

```
10          for (int i = 0; i < n; i++)
11              cin >> a[i];
12
13          for (int i = 0; i < m; i++)
14              cin >> b[i];
```

Two ordinary loops fill `a[0..n-1]` and `b[0..m-1]`. Note these arrays are **0-indexed**. That matters in the next steps.

STEP 7 The DP table, zeroed line 16

```
16          int dp[105][105] = {};
```

Meaning: `dp[i][j]` = length of the LCS of the **first i tiles** of `a` and the **first j tiles** of `b`.
Why 105? The problem guarantees n, m ≤ 100, and we need indices 0..100. So 101 is the minimum; 105 adds a small safety margin.
Why = {}? It sets every cell to 0. Row 0 and column 0 stay 0 forever, and that is the base case: an empty tower has nothing in common with anything.
Why inside the loop? It's re-created (and re-zeroed) for each test case, so old values never leak in. About 44 KB on the stack, which is fine.

STEP 8 Visit every cell, starting from 1 lines 18–19

```
18          for (int i = 1; i <= n; i++) {
19              for (int j = 1; j <= m; j++) {
```

`i` runs 1..n and `j` runs 1..m. We start at 1 because row 0 and column 0 are the pre-filled base cases. The order (row by row, left to right) guarantees that when we compute `dp[i][j]`, the cells above, to the left, and diagonally up-left are already done.

STEP 9 Case 1: the last tiles match lines 20–21

```
20                  if (a[i - 1] == b[j - 1])
21                      dp[i][j] = 1 + dp[i - 1][j - 1];
```

`dp` uses 1-based lengths, but `a` and `b` are 0-based. So the *i-th* tile of tower 1 is `a[i-1]`, and the *j-th* of tower 2 is `b[j-1]`.
If they're equal, we keep this tile in both towers. It adds 1 to the best answer for the shorter prefixes without these two tiles: `dp[i-1][j-1]`.

STEP 10 Case 2: the last tiles differ lines 22–23

```
22                  else
23                      dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
```

They can't both be the top tile of the common tower, so at least one must be thrown away. Try both options:
• drop the last tile of tower 1 → `dp[i-1][j]` (cell above)
• drop the last tile of tower 2 → `dp[i][j-1]` (cell to the left)
and keep whichever is larger with `max`.

STEP 11 Print the answer lines 28–29

```
28          cout << "Number of Tiles : " << dp[n][m] << "\n\n";
29      }
```

`tc++` is **post-increment**: it prints the current value, *then* adds 1, so the next case prints the next number.
`dp[n][m]` covers all tiles of both towers, which is the final answer. The double `\n\n` prints the blank line the problem requires after each case. Getting this wrong is a common cause of Wrong Answer / Presentation Error on UVa.

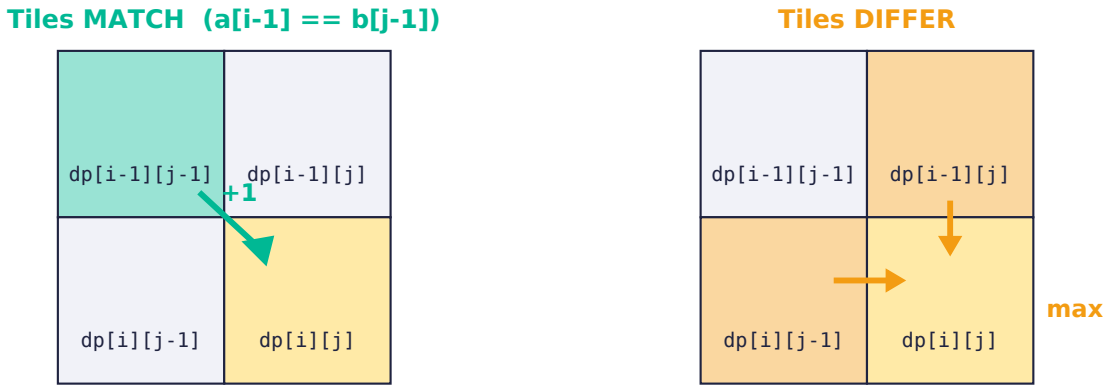
STEP 12 Done line 32

```
32  }
```

Returns 0 to signal success. In `main` it's optional (C++ adds it automatically), but harmless.

4 · The DP idea in one picture

Every cell looks only at its three neighbours: up-left, up, and left. That's why filling row by row works.




```
dp[i][j] =   1 + dp[i-1][j-1]    if a[i-1] == b[j-1]
             max(dp[i-1][j], dp[i][j-1])   otherwise
```

Why it's correct

Why is taking the match always safe? If the last tiles are equal, any optimal common tower can be rearranged to end with this pair without getting shorter. So greedily matching them never loses anything.

Why is the max enough when they differ? In any common tower, the last tile of *a* or the last tile of *b* (or both) is unused. Each choice is covered by one of the two cells, so the max covers every possibility.

5 · Filling the table by hand

Sample test case 1: a = 20 15 10 15 25 20 15, b = 15 25 10 20 15 20

Columns are tiles of **b**, rows are tiles of **a**. Each cell is `dp[i][j]`.

	j=0 empty	15 j=1	25 j=2	10 j=3	20 j=4	15 j=5	20 j=6
i=0 empty	0	0	0	0	0	0	0
20 i=1	0	0	0	0	1	1	1
15 i=2	0	**1**	**1**	1	1	2	2
10 i=3	0	1	1	**2**	**2**	2	2
15 i=4	0	1	1	2	2	**3**	3
25 i=5	0	1	2	2	2	**3**	3
20 i=6	0	1	2	2	3	3	**4**
15 i=7	0	1	2	2	3	4	**4**

match on the answer path (a tile we keep)

other matching tiles

path when backtracking

final answer dp[n][m]

A few cells traced

`dp[2][1]`	a[1]=15, b[0]=15 → match → 1 + dp[1][0] = 1 + 0 = **1**
`dp[1][1]`	a[0]=20, b[0]=15 → differ → max(dp[0][1], dp[1][0]) = max(0,0) = **0**
`dp[3][3]`	a[2]=10, b[2]=10 → match → 1 + dp[2][2] = 1 + 1 = **2**
`dp[6][4]`	a[5]=20, b[3]=20 → match → 1 + dp[5][3] = 1 + 2 = **3**
`dp[7][5]`	a[6]=15, b[4]=15 → match → 1 + dp[6][4] = 1 + 3 = **4**
`dp[7][6]`	a[6]=15, b[5]=20 → differ → max(dp[6][6], dp[7][5]) = max(3, 4) = **4** ← answer

Bonus: which tiles survive?

The code only needs the *length*, but you can recover the actual tiles by walking back from `dp[n][m]`: on a match, take the tile and move up-left; otherwise move to whichever neighbour (up or left) holds the same value.

Following the yellow/green path gives the kept tiles: **15 → 10 → 15 → 20** (bottom to top).

6 · Complexity, pitfalls, and a cheat sheet

Time	Memory
O(n·m). Each of the n·m cells is filled in O(1). With n, m ≤ 100, that's at most 10,000 steps per test case, which is instant.	**O(n·m)** for the table: 105 × 105 ints ≈ 44 KB. Reading input is O(n + m).

Memory optimization: two rows are enough

Row *i* only reads row *i−1*, so you can keep just two rows and get **O(m)** memory. Useful when n and m are large (e.g. 10^4).

```
vector<int> prev(m + 1, 0), cur(m + 1, 0);
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        if (a[i - 1] == b[j - 1]) cur[j] = 1 + prev[j - 1];
        else                      cur[j] = max(prev[j], cur[j - 1]);
    }
    swap(prev, cur);
}
// answer is prev[m]
```

Common pitfalls

Off-by-one	Comparing `a[i]` with `b[j]` instead of `a[i-1]` and `b[j-1]` skips the first tile and reads past the end of the vector.
Not resetting dp	Declaring dp outside the while loop without clearing it lets values from the previous test case leak into the next one.
Wrong blank lines	UVa expects a blank line after *every* case, including the last. Missing the second \n gives Wrong Answer.
LCS vs substring	Subsequence = can skip tiles. Substring = must be contiguous. A substring version would reset to 0 on a mismatch instead of taking the max.

Cheat sheet

State: `dp[i][j]` = LCS length of a's first i tiles and b's first j tiles.
Base: row 0 and column 0 are 0 (empty prefix).
Transition: match → 1 + up-left; differ → max(up, left).
Answer: `dp[n][m]`. **Cost:** O(n·m) time, O(n·m) or O(m) memory.
Same pattern solves: UVa 10405 (LCS), 531 (Compromise), 10100 (Longest Match), and edit-distance style problems with a small change to the transition.